



Week 1

✓ Introduction to OS (Operating Systems)

- What is OS?
 - Example of OS
 - Mobile
 - Desktop
 - Mainframe
 - Super Computer
 - Definition of OS
 - OS Goals
 - OS Components
-

✓ How an OS works?

- The Boot Process
 - Example: Linux Boot Process
 - Dual Mode Operation (Monitor, User)
 - How can we switch between 2 modes?
 - Type of Machine Instruction
-

✓ Virtual Machine

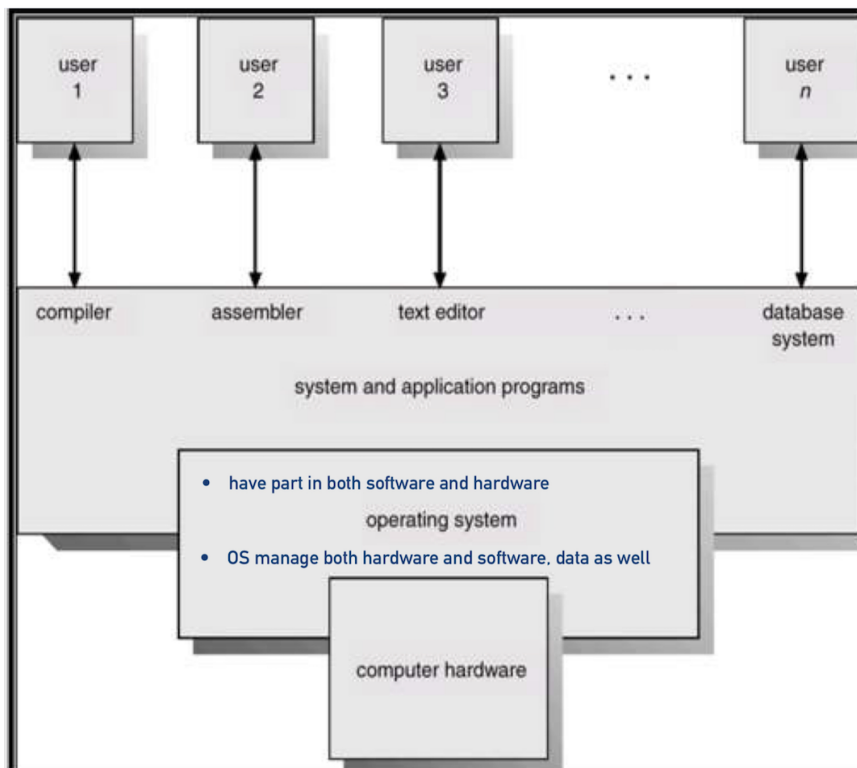
- VMware Architecture
 - Java Virtual Machine
-

| Content

✓ Introduction to OS

📌 What is Operating Systems?

- A software that manages a computer's resources.
- Control who can use and how they can use these resources.



Based on the picture, OS has both part in hardware and software. OS manage both hardware and software → **resources doesn't mean only hardware** (resources include hardware, software, and data as well). **OS manages everything!**



When a program require to use the hardware → for example, the microsoft word may need to read or write to computer hard drive → Instead of access the hard drive directly, the program request the service from OS to do that (this is 1 feature of OS to manage the hardware).



How OS manage Software → for today, we don't run only 1 program at a time (we run many program at a same time). OS need to do resource allocation for each program, for example, allocate memory for microsoft word and excel program, and then make sure that the program do not interfere with each other. (when we type in microsoft word, it should not appear in excel)

Let's look at user level

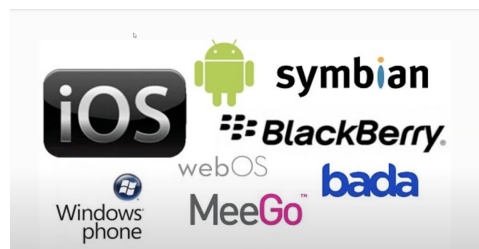
- User use the computer via the program (compiler, assembler, text editor,.....), and then the system, OS can control the user in the sense that we may set up the user account → username, password → set permission for each user account like this user is read only, read and write, read write and delete)

OS is a manager role, it's manage everything in computer system (software, hardware, data, and user)



Example of OS

- Mobile OS
 - For today, most active → IOS, Android



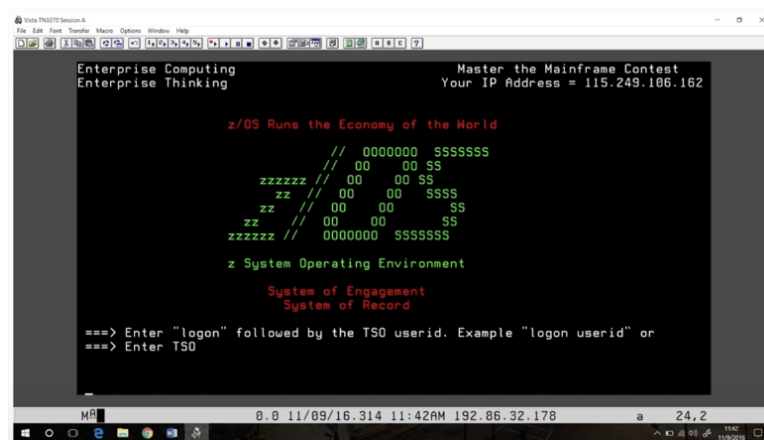
- Desktop Machine OS
 - 3 most popular OS → Windows, Mac, and Linux



- Unix-based OS
 - Many of them are Linux-distro → debian, ubuntu, redhat, CentOS
 - In our class, we use ubuntu



- Mainframe OS
 - today, majorly from IBM
 - Ex: z/Os, z/Vm, Linux for System z



- Super Computer OSes
 - majorly are Linux based

- Linux
 - specific changes to the Linux-derivative and no industry standard exists
 - http://commons.wikimedia.org/wiki/File:Operating_systems_used_on_top_500_supercomputers.svg

Definition of OS

- | | |
|---|--|
| <ul style="list-style-type: none"> • No universally accepted definition • “Everything a vendor ships when you order an operating system” is good approximation but varies wildly. | <ul style="list-style-type: none"> • However, when we study OS, we define that an OS is only its kernel- a kernel performs the most essential OS tasks. • Everything else is either a system program (ships with the operating system) or an application program. |
|---|--|

- OS can be separated into 2 parts
 - Kernel
 - Application or program that come with OS
- When we say that we going to study OS, we will only define that OS is only its kernel.
 - kernel performs the most essential OS tasks.

OS Goals

- Execute user programs and make solving user problems easier.
 - Run user program (user want to use program to solve their problem) → OS must be able to run the program that user wants. (user may want to run microsoft word for word processing → OS should be able to run it for user)
- Make the computer system convenient to use.
 - In the past, before we have OS, only the well-trained people can use the computer → because at that time we have to do many things that the OS does for us today manually (ex. load the memory, switch from 1 program to another program)

- For today, the computer system is very easy to use, because we have the OS that does those things for us.
- Use the computer hardware in an efficient manner.
 - Since we have limited resources in computer system → Good OS should manage to use this limited resource in an efficient manner (we gonna learn later how OS does this)

OS Components

we have stated previously that we going to study the essential task of OS → for each task it can be managed by the components listed below

1. Memory Manager - use to manage memory (memory = RAM)
2. Processor Manager - manage cpu of computer
3. Device Manager - manage the peripheral device (mouse, keyboard, printer, etc.)
4. File Manager - manage the file system
5. User Command Interface - will talk most about GUI, CLI
6. Network Manager - In modern OS, this is very important because OS must have the way to communicate to the network
7. Security Manager - (when we communicate to the network, we need more security)

In our class, we focus mainly on memory manager, processor manager, and user command interface



Each component has it own task to manage its resources, but there're also common task that all components need to perform.

Each component must perform the following tasks:

- Monitor its resources continuously
- Enforce the policies that determine who gets what, when, and how much
- Allocate the resource when it's appropriate
- Deallocate the resource when appropriate



Example

- Monitor its resources continuously
 - Memory Manager: monitor the RAM of computer how much space left, this part is allocated to this program, this part is allocated to this program.
- Enforce the policies that determine who gets what, when, and how much
 - File Manager: Assume each user has policies to use the system, let's say user A is allowed to use only 50MB of the space of the system. Task of File Manager is to make sure that user A never use the disk exceed 50 MB
- Allocate and Deallocate the resource when appropriate
 - Memory Manager: When the new program is launched, all the existed program need more memory, the memory manager need to find out the available space an deallocate those for the program, and when it's terminate, memory manager should do the deallocation to get the space back.

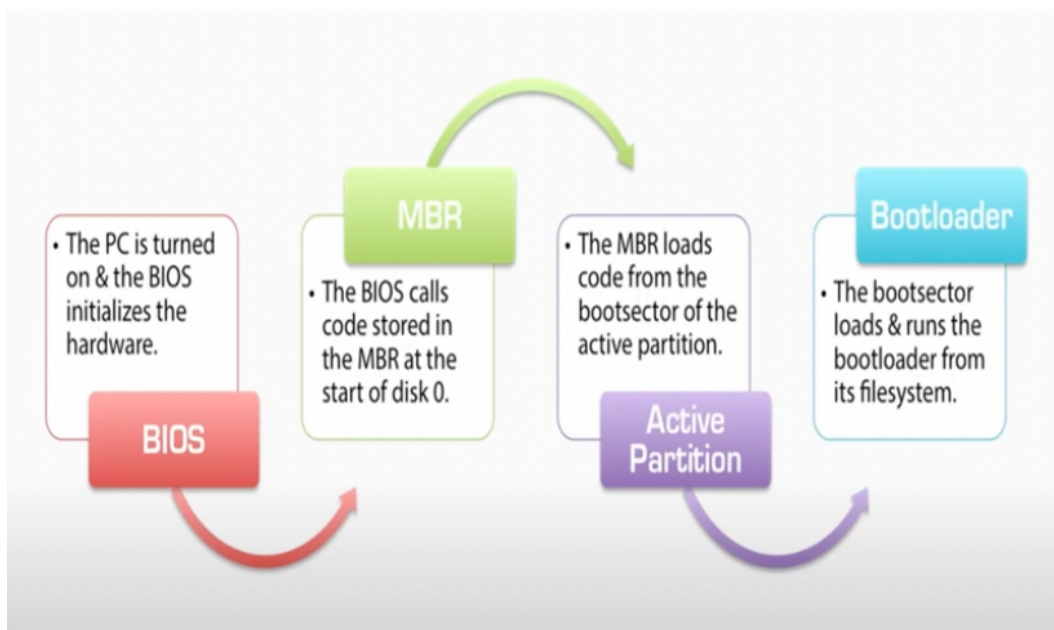
Other thing that need to understand → each component doesn't work alone, they work together → for example file manager and memory manager, when user open a file, file manager may ask memory manager to allocate the memory space for the content of a file (this allow user to access the

content of a file faster because memory work faster than storage)

✓ How an OS works?

📌 The Boot Process

- Booting - starting a computer by loading the kernel
- Process to load the kernel to memory



👤 You turn your machine on.

1. Program that store in BIOS(basic input output system) is called.
 - **The task of this program is to initialize hardware**
 - Newer Machine - UEFI (Unified Extensible Firmware Interface)

After initialize the hardware

2. Program in BIOS then called the code stored in a disk called MBR.
 - MBR - Master Boot Record
 - Newer Machine → GPT (GPID Partition Table)

3. The MBR loads code from the bootsector of the active partition
 - In 1 hard drive, you can have more than 1 active partitions
 - You can think of active partitions as partition that you can install OS. → You can install more than 1 OS per machine.
 - At the Boot Process, you can choose what OS you want to run.
4. The bootsector (from the active partition that you choose) then load and runs the bootloader
 - Bootloader - program that load kernel into memory

Linux Boot Process



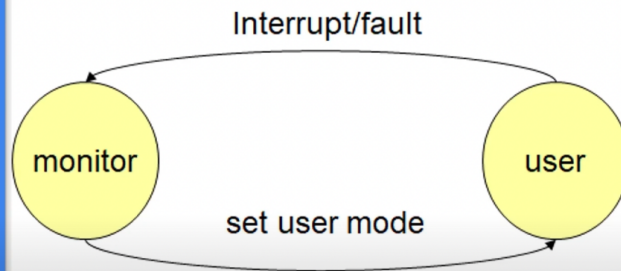
- GRUB - popular bootloader program for linux

Dual Mode Operation

- this concept can help OS protect against system error that might crash the system. This error อาจจะมาจาก bad program หรือคนที่ไม่หวังดี

Dual Mode Operation

- Mode bit added to computer hardware to indicate the current mode: monitor (0) or user (1).
- When an interrupt or fault occurs hardware switches to monitor mode.
- Privileged instructions can be issued only in monitor mode.



Computer system has 2 operation mode

1. Monitor Mode (System Mode, Privileged Mode, Kernel Mode, Supervisor Mode)
 - Mode run on behalf of the OS
2. User Mode
 - Mode run by user application program

When the system boot, the hardware start in monitor mode, and when OS is loaded, the user application run in user mode. When user application request a service from os or interrupt occur or error occur → there will be a transition from user to monitor. OS start and fulfill the request

How can we switch between 2 modes?

- In this case, we need help from hardware such as CPU
 - Cpu has mode bit 0 → monitor, 1 → user



Assuming that, user program is allow to write data anywhere in the hard drive, the problem is it may write data at the place that it already store someone else data, even worse, it may write data in the area that store info that OS need to manage hard drive. So, for this reason only OS should be able to call the machine instruction to write data to hard drive, user application if it need to write a data must ask OS and let's OS find the proper area and called an instruction to write the data.

In another way, if user application want to read date and time
→ don't have to ask OS → not dangerous.

Type of Machine Instruction

- Privileged instruction - only an OS can called them, can only be called in monitor mode.
- Non-privileged instruction - if the user program call this, don't have to switch to monitor mode.

Benefit of Dual Mode Operation

- Computer system is protected from serious error that may cause damaged to the system
- Make computer system work smoothly
 - Non-privileged can be called in user mode, Then the system don't need to pay overhead to switch back and forth between 2 modes.

Virtual Machine

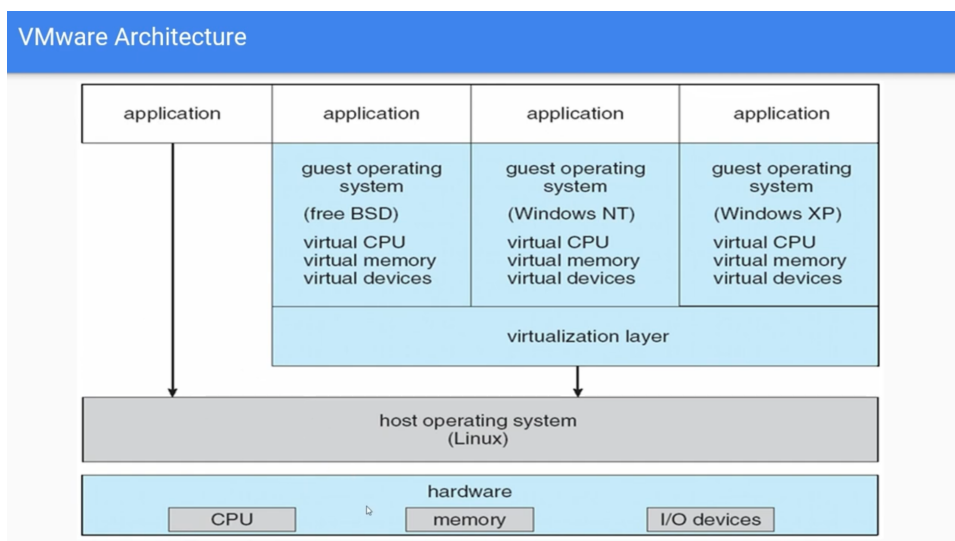
- we'll focus on virtual machine in the sense that it allows s to run more than 1 os at the same time on 1 machine.



→ Some program like web browser work like mini OS (allow more than 1 programs running under different tabs) → the point is each program running under web browser can see the resources while by the web browser → we can think that web browser create a virtual machine for those program.



VMware Architecture



- **Host OS** → OS that control the physical hardware (in pic, its Linux, but actually it can be anything)
- Then we run the **virtualization program** (Oracle Virtual Box) and then use it to create the virtual machine
 - we can specify cpu, memory, storage space inside each virtual machine, and install OS on this virtual machine.
 - In virtual machine term, the OS install and run on virtual machine is called **"guest OS"**.



We can install any OS in virtual machine, the same one, the same OS as the host OS.

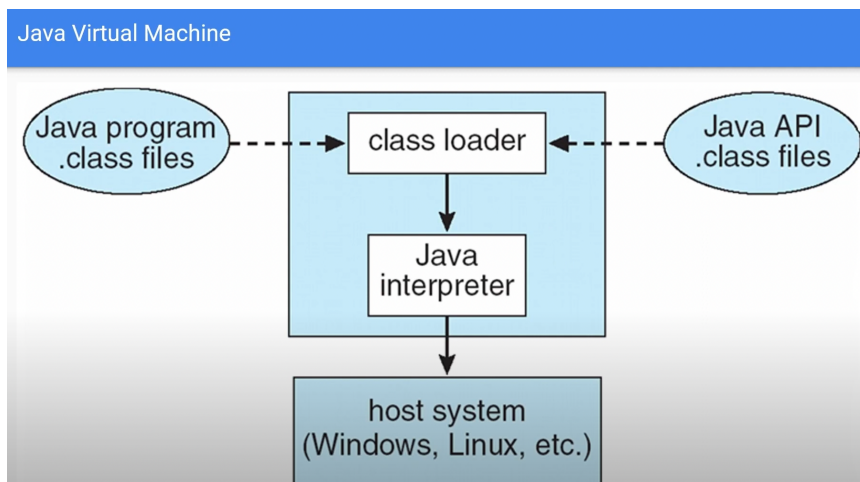
- But why we have to install the same OS as the host OS

- maybe we want to test some program and not sure that whether the program will crash the system. If it crash, it'll crash virtual machine, not physical machine.

When you install OS in the partition that you have divided in hard drive, it's not a VM concept! — you have to choose OS you want to use at boot process (choose 1 OS at the time) → VM concept, run more than 1 os at the same time.

📌 Java Virtual Machine

- in this → show how we apply the concept of Virtual Machine to programming language domain.



- Java as an example (today there are many programming languages that use this idea)

For JVM, we can think of it like guest OS, some people called it Java OS

- JVM have to run on the host OS (windows, linux, etc.)

Overview idea

1. Your Java source code will be compiled into .class file, each statement in your java code will be compiled to byte code
 2. In JVM, there are 2 main parts → class loader, java interpreter
 - class loader - load your java class and the java api that you used in your program
 - Java interpreter - read byte code line by line → when it reads byte code, it interprets the meaning of byte code and asks the host system to do the task of that byte code
-